TWINOPS ENTERPRISE

Integration Setup Guide

How GitHub, Jira, Slack and Microsoft Teams connect into your AI digital twins

Version 1.1 - October 2026
Author: Mohammad Ali

Confidential - Internal Use
Prepared for: Manager presentation and engineering reference

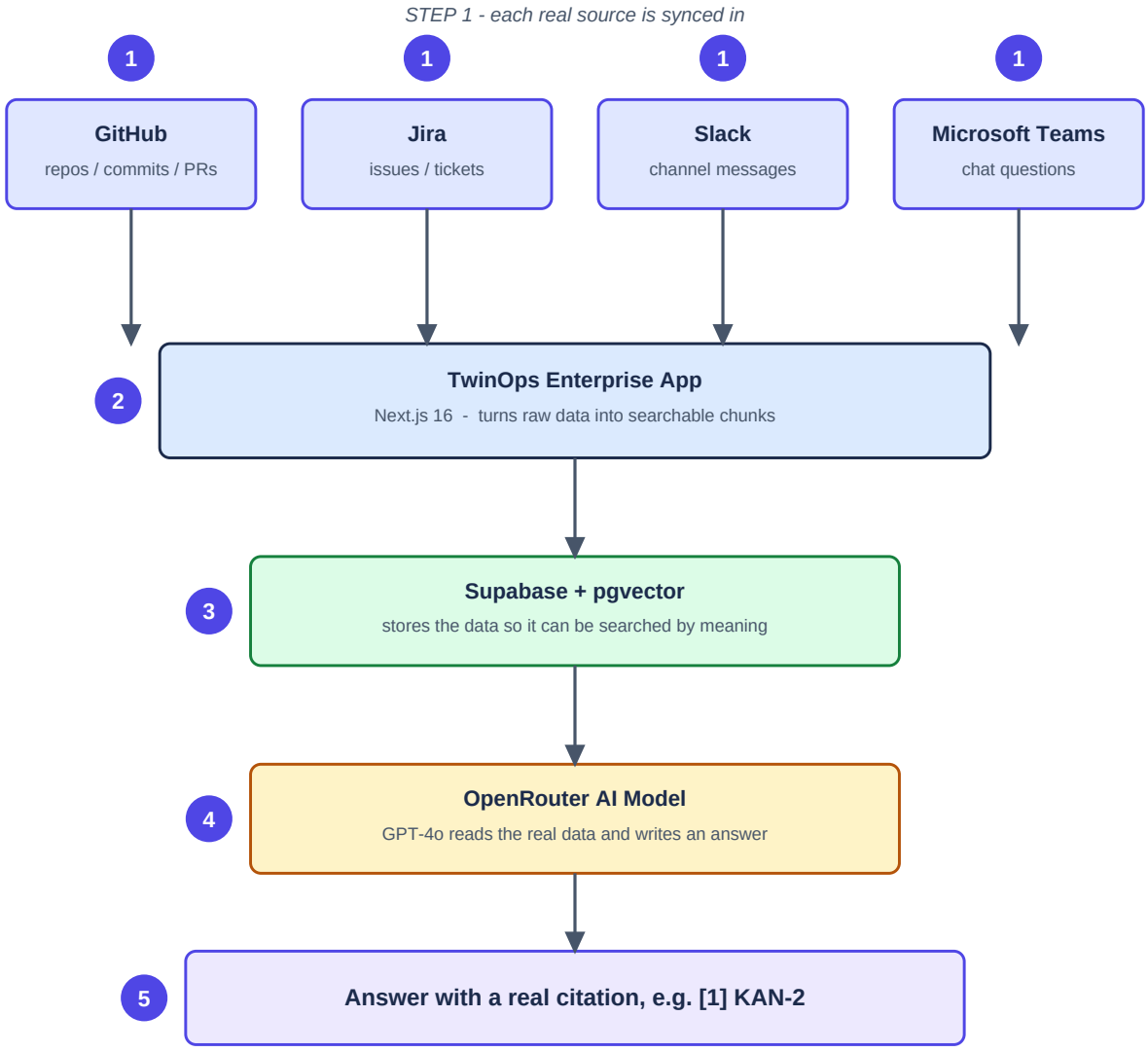
1. What TwinOps Is

TwinOps Enterprise creates an AI digital twin for each team member. A twin answers questions on that person's behalf using their own real work: GitHub repositories, Jira tickets, Slack conversations and Microsoft Teams messages. Every answer is grounded in real synced data and includes a citation back to the source, so nothing is invented.

This document explains, in plain language and in order, how each integration connects, exactly which button to click and where, what the AI model actually does with your data, the security controls already in place, and the current status of the project.

2. How It Actually Works (The AI Model Explained)

This is the part people usually skip, but it matters most. Follow the five numbered steps below, in order, left side of each box tells you which step it is.



Step 1. Real data goes in. GitHub, Jira, Slack and Microsoft Teams are the four real sources. Nothing here is sample data, it is your actual repos, tickets and messages.

Step 2. The app breaks data into small pieces. A Jira ticket or a GitHub file is too big to search in one go, so TwinOps cuts it into small overlapping pieces called chunks, about 700 characters each.

Step 3. Each piece is stored so it can be found by meaning. Every chunk is turned into a list of numbers called an embedding, and saved in a database called Supabase. This lets the system find relevant pieces even if the exact

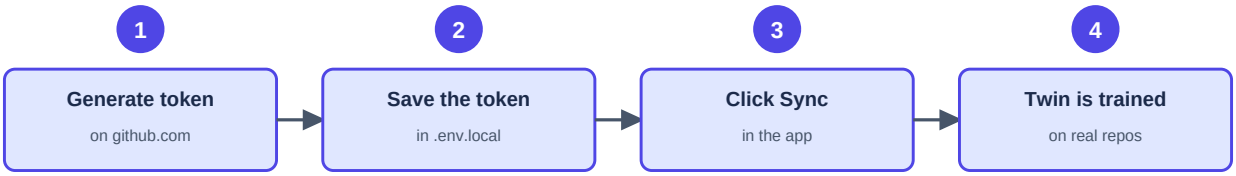
words do not match.

Step 4. The AI model reads only the relevant pieces and writes an answer. TwinOps sends the matching chunks, and only the matching chunks, to an AI model called GPT-4o (reached through a service called OpenRouter). The model is told clearly: only answer from what is given, never make things up.

Step 5. The answer comes back with a real citation. Every answer points back to the exact Jira ticket, GitHub file, or message it came from, so you can always check the source yourself.

3. Connecting GitHub

Current status: working, already connected with real data.



Step 1. Open GitHub and create a token.

Go to github.com, click your profile picture, then Settings, then Developer settings (bottom of the left menu), then Personal access tokens, then Fine-grained tokens.

Step 2. Set the right permissions.

Give the token read-only access to Repository contents and Metadata. Do not give write access, it is not needed.

Step 3. Copy the token into the project.

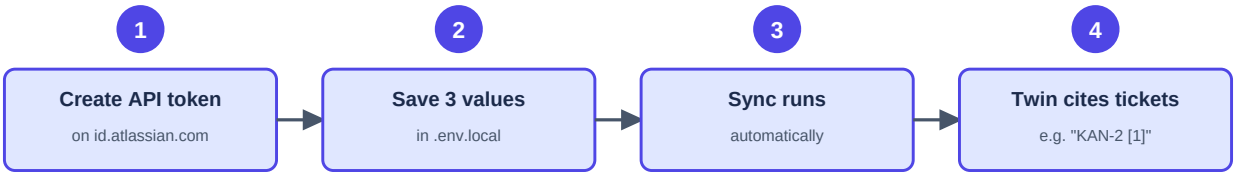
Open the file .env.local in the twinops-enterprise folder and add this line: GITHUB_TOKEN=your_token_here

Step 4. Run the sync.

Open the app, go to your twin, and click Sync. Your real repos, commits and pull requests are read, broken into chunks, and stored.

4. Connecting Jira

Current status: working, already connected with real data.



Step 1. Open Atlassian account settings.

Go to id.atlassian.com, click your profile picture, then Manage account, then the Security tab.

Step 2. Create an API token.

Under API tokens, click Create and manage API tokens, then Create API token. Give it a name like TwinOps and copy the value shown.

Step 3. Add three values to .env.local.

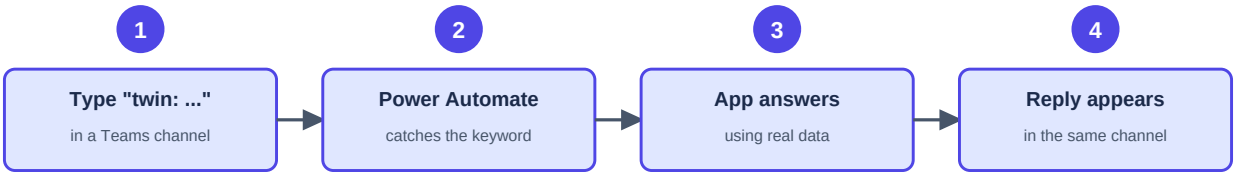
JIRA_BASE_URL (for example https://yourname.atlassian.net), JIRA_EMAIL (your login email), and JIRA_API_TOKEN (the value you just copied).

Step 4. Sync and check.

Click Sync in the app. Jira ticket text (which is stored in a rich format called ADF) is automatically converted to plain text, broken into chunks, and stored. Running sync again safely replaces old data instead of duplicating it.

5. Connecting Microsoft Teams

Current status: working, tested end to end in a real Teams channel.



Step 1. Someone types a question in Teams.

The message must contain the word "twin:" followed by the question, for example: twin: what is the status of KAN-2?

Step 2. A Power Automate flow catches it.

This flow was built once in advance at make.powerautomate.com. It watches the channel for the keyword, then sends the question to the TwinOps app over the internet.

Step 3. The app checks a secret key, then answers.

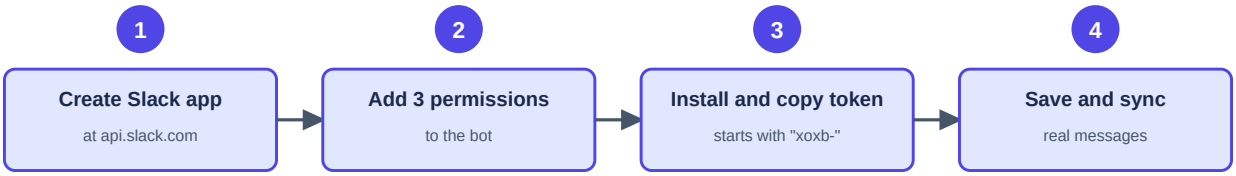
Every request must include a secret password-like value so random people on the internet cannot use this endpoint. Once verified, the app runs the exact same search-and-answer process described in Section 2.

Step 4. The answer is posted back in Teams.

Power Automate takes the answer the app generated and posts it as a reply in the same Teams conversation, with the real citation included.

6. Connecting Slack

Current status: all the code is ready and tested. Only the final bot token is missing.



Step 1. Create a Slack app.

Go to api.slack.com/apps, click "Create New App", choose "From scratch", name it TwinOps, and pick your workspace.

Step 2. Give the bot permission to read messages.

Inside the app, click "OAuth and Permissions" on the left menu. Under "Bot Token Scopes" add exactly three: channels:history, channels:read, users:read.

Step 3. Install the app and copy the token.

Scroll to the top of the same page and click "Install to Workspace", then "Allow". A value appears labeled "Bot User OAuth Token" starting with xoxb-. This is the one and only token we need, copy it.

Step 4. Save the token and sync.

Add it to .env.local as SLACK_BOT_TOKEN. In Slack, invite the bot into a real channel by typing /invite @TwinOps. Then click Sync in the app to pull in real conversation history.

7. Security Controls Already In Place

These are not planned for later, they are already written and working in the code today.

Control	What it actually does
Row Level Security (RLS)	The database is locked down. No outside or anonymous user can read or write data directly, only
Protection against internal address abuse (SSRF)	Blocks attempts to point the Jira or Teams connection at an internal or hidden address instead of
Secret key check for Teams	The Teams endpoint only accepts requests that include the correct secret key, checked in a way t
No real secrets stored on GitHub	All real API keys and tokens live only on this computer in .env.local, which is excluded from GitHu

One known and accepted gap: creating, editing, or deleting a twin currently does not require logging in. This was chosen on purpose to keep the current demo simple, it was not missed by mistake.

8. Current Status, At A Glance

Area	Status
GitHub sync	**Working with real data**
Jira sync	**Working with real data**
Microsoft Teams	**Working, tested end to end**
Slack	**Ready, final bot token pending**
Search that understands meaning (RAG)	**Working**
AI answers with real citations	**Working**
One twin asking another twin for help	**Working**
Database security (RLS)	**Done**
Protection against internal address abuse	**Done**
Saving uploaded documents	**Not yet saving data, low priority**
Remembering past conversations	**Not yet saved, low priority**
Login before managing twins	**Left out on purpose for now**

End of document. For a longer walkthrough with troubleshooting tips, see docs/SETUP-GUIDE.md in the project folder.